

The Right Denoising Order Is a Doob Transform: Harmonic Test-Time Scaling for Diffusion Language Models

Proposal draft

May 22, 2026

Abstract

Diffusion language models (DLMs) expose a degree of freedom that autoregressive language models do not: at inference time, they can decide *which* token or block to denoise next. Existing test-time scaling methods exploit extra compute by sampling more trajectories, refining full trajectories, or resampling promising search frontiers. These methods are effective, but they leave a more fundamental question implicit: *what is the optimal reward-conditioned transition kernel on the DLM denoising process?* This proposal argues that the answer is classical and exact: the optimal reward-conditioned denoising process is the Doob h -transform of the base DLM path measure, where $h_t(x_t) = \mathbb{E}[\exp(\beta R(X_0)) \mid X_t = x_t]$ is the backward information function. On the mask lattice of a masked DLM, this transform induces a principled denoising-order rule: reveal the edge whose child state has the largest harmonic future-reward advantage, not the token with the highest local confidence. We propose HDO-DLM, a training-free inference-time algorithm that estimates h_t by Bellman-calibrated verifier rollouts and uses the resulting h -ratio to select token/block denoising moves. The central view is that test-time scaling for DLMs should be understood as progressively solving a backward harmonic equation on the mask lattice.

1 One-line thesis

Do not denoise the easiest token. Denoise the edge with the largest harmonic future-reward advantage.

The proposed method is not another search heuristic. It starts from an exact identity: if the desired inference-time distribution is a reward-tilted version of the base DLM distribution, then the optimal controlled denoising process is the Doob h -transform of the base denoising chain. The missing object is the backward information function h_t . The extra test-time compute should therefore be spent estimating h_t and applying its ratio to choose denoising transitions.

2 Motivation: what is missing in current DLM test-time scaling?

Let c denote the prompt and $x_0 = (x_{0,1}, \dots, x_{0,L})$ the generated response. A reward or verifier $R(x_0, c)$ measures task quality: correctness, reasoning quality, preference, toxicity control, or a process-reward score. The standard KL-regularized inference-time target is

$$\pi_\beta(x_0 \mid c) \propto p_\theta(x_0 \mid c) \exp\{\beta R(x_0, c)\}, \quad (1)$$

where p_θ is the base DLM marginal and $\beta > 0$ controls reward strength.

Recent methods differ mainly in how they approximate (1):

- **Best-of- K** samples K independent generations from p_θ and chooses the best. It never changes the generative path measure, so it is limited when high-probability regions are misaligned with high-reward regions.
- **SMC / Feynman–Kac steering** reweights intermediate particles by partial potentials, but often suffers weight degeneracy and performs a single backward pass.
- **PG-DLM** performs trajectory-level refinement by constructing a Markov chain over complete denoising trajectories and applying a conditional SMC kernel. Its key idea is to preserve a reference trajectory while resampling alternative trajectories around it.
- S^3 uses verifier-guided search during denoising. It expands frontier candidates, estimates their final quality with one-step clean prediction, and resamples promising frontier states while preserving diversity. It can be interpreted as an approximate reward-tilted or twisted search process.

These are important advances, but they share a hidden gap: they approximate the reward-conditioned process without explicitly solving for the *optimal reward-conditioned transition*. In a Markov process, that transition is not arbitrary. It has a closed form.

The hidden object. The correct object is the backward information function

$$h_t(x_t) := \mathbb{E}_{p_\theta} [\exp\{\beta R(X_0, c)\} \mid X_t = x_t, c]. \quad (2)$$

It is the expected future terminal reward potential from a partially denoised state. If we knew h_t exactly, we could sample from the reward-tilted path measure exactly by modifying every DLM transition with an h -ratio.

Core critique. Current methods spend compute sampling, scoring, and resampling trajectories. HDO-DLM spends compute estimating the missing harmonic function that defines the optimal reward-conditioned denoising dynamics.

3 Background: DLMs as discrete denoising chains

3.1 DLM path measure

We model a discrete diffusion language model as a reverse-time Markov chain

$$X_T \rightarrow X_{T-1} \rightarrow \cdots \rightarrow X_0, \quad (3)$$

where X_T is a noisy or fully masked state and X_0 is the final sequence. The base reverse transition is

$$K_t(x_{t-1} \mid x_t, c), \quad t = T, T-1, \dots, 1. \quad (4)$$

The path distribution is

$$p_\theta(x_{0:T} \mid c) = p_T(x_T \mid c) \prod_{t=T}^1 K_t(x_{t-1} \mid x_t, c). \quad (5)$$

For masked diffusion LMs, X_t often lives on a mask lattice: each state specifies which positions are visible and which remain masked. This non-autoregressive structure is crucial because it allows the sampler to choose denoising order.

3.2 Mask lattice specialization

Let $[L] = \{1, \dots, L\}$ index response positions, and let $S \subseteq [L]$ denote the set of already revealed positions. A state on the mask lattice is

$$x_S = \{x_i : i \in S\} \cup \{\text{[MASK]} : i \notin S\}. \quad (6)$$

A one-token reveal transition chooses a masked position $i \notin S$ and a token $v \in \mathcal{V}$:

$$x_{S \cup \{i\}} = x_S^{i=v}. \quad (7)$$

A generic base DLM edge kernel can be factored as

$$K(S, i, v \mid x_S, c) = q_t(i \mid x_S, c) p_\theta(v \mid x_S, i, c), \quad (8)$$

where $q_t(i \mid x_S, c)$ is the base position schedule and $p_\theta(v \mid x_S, i, c)$ is the denoiser distribution for token i . Many practical samplers use confidence-based schedules, e.g.

$$i^{\text{conf}} = \arg \max_{i \notin S} \max_v p_\theta(v \mid x_S, i, c). \quad (9)$$

The claim of this proposal is that confidence is not the mathematically correct criterion for reward-conditioned inference. The correct criterion comes from the Doob transform.

4 Reward-tilted path measure

4.1 From KL-regularized generation to reward tilting

Consider the KL-regularized objective over candidate inference-time distributions q :

$$\max_q \mathbb{E}_{q(x_0|c)}[R(x_0, c)] - \frac{1}{\beta} \text{KL}(q(x_0 \mid c) \parallel p_\theta(x_0 \mid c)). \quad (10)$$

A standard variational calculation gives

$$q^*(x_0 \mid c) = \frac{1}{Z_0(c)} p_\theta(x_0 \mid c) \exp\{\beta R(x_0, c)\}, \quad (11)$$

where

$$Z_0(c) = \mathbb{E}_{p_\theta(x_0|c)}[\exp\{\beta R(X_0, c)\}]. \quad (12)$$

Because $p_\theta(x_0 \mid c)$ is induced by the DLM path measure (5), the natural lifted target over paths is

$$\Pi_\beta(x_{0:T} \mid c) = \frac{1}{Z(c)} p_\theta(x_{0:T} \mid c) G(x_0, c), \quad (13)$$

where

$$G(x_0, c) = \exp\{\beta R(x_0, c)\}. \quad (14)$$

4.2 Why terminal rewards create a sequential inference problem

The reward potential G is terminal. During denoising, at state x_t , the sampler does not know the final x_0 yet. This creates a smoothing problem: how should a transition at time t account for a reward that will only be evaluated at time 0?

The answer is the conditional expectation of the future terminal potential:

$$h_t(x_t) = \mathbb{E}[G(X_0, c) \mid X_t = x_t, c]. \quad (15)$$

This is the same conceptual role as a twist function in twisted SMC or a desirability function in linearly-solvable control. It is also the harmonic function used in Doob conditioning.

5 Backward information function and harmonic equation

Definition 1 (Backward information function). *For the base DLM chain (5) and terminal potential $G(x_0, c) > 0$, define*

$$h_t(x_t) := \mathbb{E}_{p_\theta} [G(X_0, c) \mid X_t = x_t, c], \quad t = 0, 1, \dots, T. \quad (16)$$

At $t = 0$, $h_0(x_0) = G(x_0, c)$.

Proposition 1 (Backward harmonic equation). *For $t = 1, \dots, T$, the backward information function satisfies*

$$h_t(x_t) = \sum_{x_{t-1} \in \mathcal{X}} K_t(x_{t-1} \mid x_t, c) h_{t-1}(x_{t-1}). \quad (17)$$

Equivalently,

$$h_t(X_t) = \mathbb{E}[h_{t-1}(X_{t-1}) \mid X_t] \quad (18)$$

under the base DLM process.

Proof. By the tower property,

$$h_t(x_t) = \mathbb{E}[G(X_0, c) \mid X_t = x_t, c] \quad (19)$$

$$= \mathbb{E}[\mathbb{E}[G(X_0, c) \mid X_{t-1}, c] \mid X_t = x_t, c] \quad (20)$$

$$= \sum_{x_{t-1}} K_t(x_{t-1} \mid x_t, c) h_{t-1}(x_{t-1}). \quad (21)$$

□

Proposition 2 (Martingale property). *Under the base DLM process, $\{h_t(X_t)\}_{t=0}^T$ is a reverse-time martingale in the denoising direction:*

$$\mathbb{E}[h_{t-1}(X_{t-1}) \mid X_t] = h_t(X_t). \quad (22)$$

This martingale identity is more than a mathematical curiosity. It gives a diagnostic for the quality of any approximate $\hat{h}_t$. If $\hat{h}_t$ is a good backward information estimate, then the Bellman residual

$$\mathcal{E}_t(x_t) = \left| \log \hat{h}_t(x_t) - \log \left(\frac{1}{M} \sum_{m=1}^M \hat{h}_{t-1}(Y_m) \right) \right|, \quad Y_m \sim K_t(\cdot \mid x_t, c), \quad (23)$$

should be small. This gives a natural adaptive-compute rule: spend more test-time compute where the harmonic residual is high.

6 Exact reward-conditioned denoising via Doob transform

Theorem 1 (Doob-transformed DLM transition). *Let h_t be defined by (2). Assume $h_t(x_t) > 0$ for all states reachable under the base DLM. Define*

$$K_t^*(x_{t-1} \mid x_t, c) = K_t(x_{t-1} \mid x_t, c) \frac{h_{t-1}(x_{t-1})}{h_t(x_t)}. \quad (24)$$

Then K_t^ is a valid Markov transition kernel, and the path measure generated by $p_T^*(x_T) \propto p_T(x_T) h_T(x_T)$ and transitions K_t^* equals the reward-tilted path measure (13).*

Proof. First, normalization follows from the harmonic equation:

$$\sum_{x_{t-1}} K_t^*(x_{t-1} \mid x_t, c) = \frac{1}{h_t(x_t)} \sum_{x_{t-1}} K_t(x_{t-1} \mid x_t, c) h_{t-1}(x_{t-1}) \quad (25)$$

$$= 1. \quad (26)$$

Now consider the path probability under the transformed process:

$$p^*(x_{0:T} \mid c) = p_T^*(x_T \mid c) \prod_{t=T}^1 K_t^*(x_{t-1} \mid x_t, c) \quad (27)$$

$$= \frac{p_T(x_T) h_T(x_T)}{Z(c)} \prod_{t=T}^1 \left[K_t(x_{t-1} \mid x_t, c) \frac{h_{t-1}(x_{t-1})}{h_t(x_t)} \right] \quad (28)$$

$$= \frac{1}{Z(c)} p_T(x_T) \prod_{t=T}^1 K_t(x_{t-1} \mid x_t, c) h_0(x_0) \quad (29)$$

$$= \frac{1}{Z(c)} p_\theta(x_{0:T} \mid c) G(x_0, c), \quad (30)$$

which is exactly (13). $\square$

Interpretation. The Doob transform says:

$$\boxed{\text{optimal controlled transition} = \text{base DLM transition} \times \text{future reward advantage.}} \quad (31)$$

The future reward advantage of child state x_{t-1} relative to parent x_t is

$$A_t(x_{t-1}; x_t) = \frac{h_{t-1}(x_{t-1})}{h_t(x_t)}. \quad (32)$$

A transition should be amplified if, after taking it, the expected terminal reward potential increases.

7 Denoising order as harmonic flow on the mask lattice

Now specialize the Doob kernel to a masked DLM state x_S . A child edge is (i, v) , where $i \notin S$ and $v \in \mathcal{V}$:

$$y = x_S^{i=v} = x_{S \cup \{i\}}. \quad (33)$$

The Doob-transformed edge probability is

$$K^*(i, v \mid x_S, c) = K(i, v \mid x_S, c) \frac{h(x_S^{i=v})}{h(x_S)}. \quad (34)$$

Using the factorization (8),

$$K^*(i, v \mid x_S, c) \propto q(i \mid x_S, c) p_\theta(v \mid x_S, i, c) h(x_S^{i=v}). \quad (35)$$

The induced optimal position-order distribution is obtained by marginalizing out v :

$$P^*(i \mid x_S, c) \propto q(i \mid x_S, c) \sum_{v \in \mathcal{V}} p_\theta(v \mid x_S, i, c) h(x_S^{i=v}). \quad (36)$$

Key consequence. Confidence-based denoising uses

$$i^{\text{conf}} = \arg \max_{i \notin S} \max_v p_\theta(v \mid x_S, i, c). \quad (37)$$

Doob denoising uses

$$i^{\text{Doob}} = \arg \max_{i \notin S} q(i \mid x_S, c) \mathbb{E}_{v \sim p_\theta(\cdot \mid x_S, i, c)} [h(x_S^{i=v})]. \quad (38)$$

Therefore the optimal denoising order is not easy-to-hard. It is high-harmonic-advantage first.

8 Why confidence-based ordering is structurally wrong

A high-confidence token is a token whose local marginal is peaked. But a good denoising action is not merely a locally easy decision; it is an action that moves the state toward high terminal reward.

8.1 A minimal counterexample

Suppose two masked positions remain: a and b . Position a is easy but irrelevant; position b is uncertain but determines final correctness. Let

$$\max_v p_\theta(v \mid x_S, a) = 0.99, \quad \max_v p_\theta(v \mid x_S, b) = 0.55. \quad (39)$$

A confidence schedule denoises a . But suppose revealing a leaves expected future reward unchanged:

$$\mathbb{E}[h(x_S^{a=v})] \approx h(x_S), \quad (40)$$

while revealing b has high future-reward separation:

$$\mathbb{E}[h(x_S^{b=v})] \gg h(x_S). \quad (41)$$

Then (38) selects b , not a . This is exactly the reasoning-task regime: filler tokens are often easy; structural tokens, operators, variables, intermediate numbers, and logical connectives are less certain but more decision-relevant.

8.2 Information view

For small β , expand the terminal potential:

$$G(x_0) = \exp\{\beta R(x_0)\} = 1 + \beta R(x_0) + O(\beta^2). \quad (42)$$

Then

$$h(x_S^{i=v}) = 1 + \beta \mathbb{E}[R(X_0) \mid x_S^{i=v}] + O(\beta^2). \quad (43)$$

The Doob score for position i becomes

$$\mathbb{E}_{v \sim p_\theta} [h(x_S^{i=v})] = 1 + \beta \mathbb{E}_{v \sim p_\theta} \mathbb{E}[R(X_0) \mid x_S^{i=v}] + O(\beta^2). \quad (44)$$

Thus, in the weak-reward regime, Doob ordering chooses the token whose reveal maximizes expected reward-to-go, not local confidence. In the strong-reward regime, the exponential potential emphasizes rare high-reward continuations.

9 The proposed method: HDO-DLM

9.1 Overview

HDO-DLM is a training-free inference-time algorithm for DLMs. It has three components:

1. **Candidate edge expansion:** from a partially denoised state, propose candidate denoising edges using the base DLM.
2. **Harmonic evaluation:** estimate h for each child state using verifier-based clean prediction and Bellman backups.
3. **Doob ordering:** select or sample the next denoising edge with probability proportional to the base transition multiplied by estimated future-reward advantage.

The method is called “harmonic” because h_t satisfies the backward harmonic equation (17).

9.2 Clean-prediction initialization

Given an intermediate child state $y = x_{t-1}$, define a deterministic or stochastic clean prediction operator

$$\widehat{x}_0(y) = \text{CleanPredict}_\theta(y, c), \quad (45)$$

e.g. by filling remaining masks with the model’s current prediction. A simple initial estimator is

$$\hat{h}_{t-1}^{(0)}(y) = \exp \{ \beta V(\widehat{x}_0(y), c) \}, \quad (46)$$

where V is a verifier or PRM. This is close to the kind of lookahead used in S^3 , but it is only the initialization in HDO-DLM.

9.3 Bellman-calibrated harmonic backup

The estimator (46) is not guaranteed to satisfy the harmonic equation. We refine it by local Bellman backups. For a state x_t , draw M children

$$Y_m \sim K_t(\cdot \mid x_t, c), \quad m = 1, \dots, M. \quad (47)$$

Then the Bellman target is

$$\mathcal{B}\hat{h}_t(x_t) = \frac{1}{M} \sum_{m=1}^M \hat{h}_{t-1}(Y_m). \quad (48)$$

In log-space, for numerical stability,

$$\widehat{\ell}_t(x_t) = \log \left(\frac{1}{M} \sum_{m=1}^M \exp \{ \widehat{\ell}_{t-1}(Y_m) \} \right), \quad \widehat{\ell} = \log \widehat{h}. \quad (49)$$

A practical smoothed update is

$$\widehat{\ell}_t^{(d+1)}(x) = (1 - \alpha) \widehat{\ell}_t^{(0)}(x) + \alpha \log \left(\frac{1}{M} \sum_{m=1}^M \exp \{ \widehat{\ell}_{t-1}^{(d)}(Y_m) \} \right). \quad (50)$$

Here d is the number of harmonic backup iterations. This creates a new test-time scaling axis: more compute means a better approximation to the backward information function.

9.4 Doob edge score

For a candidate child edge $y_b \sim K_t(\cdot \mid x_t, c)$, define the estimated Doob score

$$s_b = \log K_t(y_b \mid x_t, c) + \hat{\ell}_{t-1}(y_b). \quad (51)$$

If candidates are sampled from K_t , the proposal term can be omitted for resampling among proposed candidates:

$$s_b \propto \hat{\ell}_{t-1}(y_b). \quad (52)$$

The next child is sampled as

$$P(y_b \mid x_t, c) = \frac{\exp(s_b/\tau)}{\sum_{b'} \exp(s_{b'}/\tau)}, \quad (53)$$

where τ is an exploration temperature. Greedy decoding uses $\tau \rightarrow 0$.

9.5 Adaptive compute by martingale residual

The Bellman residual at a state x_t is

$$\hat{\mathcal{E}}_t(x_t) = \left| \hat{\ell}_t(x_t) - \log \left(\frac{1}{M} \sum_{m=1}^M \exp\{\hat{\ell}_{t-1}(Y_m)\} \right) \right|. \quad (54)$$

If $\hat{\mathcal{E}}_t(x_t)$ is large, then the current clean-prediction estimate disagrees with the rollout-implied future reward. HDO-DLM allocates more backup samples or deeper rollouts at such states. This mirrors the adaptive-compute spirit of PG-DLM, but the trigger is a principled harmonic-consistency error.

10 Algorithm

10.1 Main algorithm

Algorithm 1: HDO-DLM – Harmonic Doob Ordering for DLMs

Input:

DLM reverse kernels K_t
prompt c
verifier / reward model V
reward temperature β
particle count N
candidate branching B
harmonic backup width M
maximum backup depth $D_{\max}$
residual threshold ϵ

Initialize particles $x_{T^1}, \dots, x_{T^N}$ as fully masked states.

```
for  $t = T, T-1, \dots, 1$  do
  for each particle  $n = 1, \dots, N$  do
    1. Candidate expansion:
      sample candidate children  $y_b \sim K_t(. \mid x_{t^{\wedge}n}, c)$ ,  $b=1, \dots, B$ .

    2. Initial h-estimation:
      for each candidate  $y_b$ :
         $\hat{x}_0 = \text{CleanPredict\_theta}(y_b, c)$ 
         $\ell_0(y_b) = \beta * V(\hat{x}_0, c)$ 

    3. Bellman calibration:
      repeat  $d = 1, \dots, D_{\max}$ :
        for each candidate/state  $y_b$ :
          sample  $M$  rollout children  $z_m \sim K_{\{t-1\}}(. \mid y_b, c)$ 
           $\ell_d(y_b) = \text{logmeanexp}_m \ell_{\{d-1\}}(z_m)$ 
          compute martingale residual  $E_{\hat{}}$ 
          if  $E_{\hat{}} < \epsilon$ : break

    4. Doob edge selection:
       $\text{score}_b = \log K_t(y_b \mid x_{t^{\wedge}n}, c) + \ell_d(y_b)$ 
      choose  $y_{\text{star}}$  by  $\text{softmax}(\text{score}_b / \tau)$ 

    5. Update particle:
       $x_{\{t-1\}^{\wedge}n} = y_{\text{star}}$ 
```

Optional population resampling:

resample particles using $\exp(\ell(x_{\{t-1\}^{\wedge}n}))$ if ESS is low.

Return final samples $x_0^1, \dots, x_0^N$ or the verifier-best output.

10.2 Position-level variant

For masked DLMs that reveal multiple tokens per denoising step, HDO-DLM can first score positions and then sample token values. For each position $i \notin S$, take top- K candidate tokens $v \in \mathcal{T}_i$. Define

$$\hat{H}_i(x_S) = \sum_{v \in \mathcal{T}_i} p_\theta(v \mid x_S, i, c) \hat{h}(x_S^{i=v}). \quad (55)$$

Then choose positions by

$$P(i \mid x_S, c) = \frac{q(i \mid x_S, c) \hat{H}_i(x_S)}{\sum_{j \notin S} q(j \mid x_S, c) \hat{H}_j(x_S)}. \quad (56)$$

After selecting i , sample or greedily choose

$$P(v \mid i, x_S, c) \propto p_\theta(v \mid x_S, i, c) \hat{h}(x_S^{i=v}). \quad (57)$$

This jointly modifies *where-to-denoise* and *what-to-denoise* according to the same Doob principle.

10.3 Block-level variant

If revealing single tokens is too expensive, use blocks $B \subseteq [L] \setminus S$ such as equation spans, sentence fragments, code lines, or groups of high-mutual-information tokens. For a block proposal $y = x_S^{B=u_B}$,

$$K^*(B, u_B \mid x_S) \propto K(B, u_B \mid x_S) \hat{h}(x_S^{B=u_B}). \quad (58)$$

This makes HDO-DLM compatible with practical DLM decoding schedules that update multiple tokens at once.

11 Relationship to SMC, PG-DLM, S^3 , and twisted SMC

11.1 SMC and Feynman–Kac correctors

SMC methods approximate sequences of distributions by mutation, weighting, and resampling. In reward-guided DLMs, the terminal reward induces a Feynman–Kac path measure. The ideal incremental correction is not a raw terminal reward applied late; it is the ratio of future potentials propagated backward through the chain. HDO-DLM uses this fact to turn Feynman–Kac correction into a denoising-order rule.

11.2 PG-DLM

PG-DLM introduces a powerful scaling axis: the number of trajectory-refinement iterations. It fixes a reference trajectory and uses conditional SMC to resample alternative complete trajectories. In contrast, HDO-DLM changes the transition rule inside a denoising trajectory. It asks: before running many trajectory-level refinements, can we make each transition closer to the reward-conditioned transition?

	PG-DLM	HDO-DLM
Main object	full denoising trajectory $x_{0:T}$	backward information function h_t
Scaling axis	particle Gibbs iterations	harmonic backup accuracy/depth
Preserved structure	reference trajectory	Doob-transformed reward flow
Transition change	via conditional SMC refinement	direct h -ratio edge weighting

11.3 S^3

S^3 uses verifier-guided trajectory expansion and frontier resampling. A simplified weight can be seen as

$$w_{S^3}(y) \propto \exp\{\beta V(\hat{x}_0(y), c)\}. \quad (59)$$

This is a useful one-step lookahead. HDO-DLM interprets this as an initial estimate of $h(y)$, then improves it by Bellman calibration and uses it as an h -ratio in a Doob edge kernel.

$$\begin{aligned} S^3: & \text{ verifier lookahead as a frontier score,} \\ \text{HDO-DLM:} & \text{ verifier lookahead as an approximate harmonic function.} \end{aligned} \quad (60)$$

11.4 Twisted SMC for language models

Twisted SMC uses learned twist functions to estimate future potentials and focus computation on promising partial sequences. HDO-DLM is inspired by the same mathematical principle, but its contribution is DLM-specific: it applies the twist on the mask lattice, where the twist directly defines which position or block to denoise next. Autoregressive LMs cannot freely choose this edge because their next edge is fixed by textual order.

12 Theoretical properties

12.1 Exactness with exact h

Theorem 2 (Exactness). *If HDO-DLM uses the exact backward information function h_t and samples transitions from (24), then its final marginal distribution is exactly the reward-tilted target (11).*

Proof. This follows immediately from the Doob-transform theorem and marginalization of the path target (13) over $x_{1:T}$. $\square$

12.2 Approximation error

Let $\hat{h}_t$ be an approximation. Define multiplicative error

$$e_t(x) = \frac{\hat{h}_t(x)}{h_t(x)}. \quad (61)$$

The approximate Doob transition is

$$\hat{K}_t(x_{t-1} | x_t) = K_t(x_{t-1} | x_t) \frac{\hat{h}_{t-1}(x_{t-1})}{\sum_y K_t(y | x_t) \hat{h}_{t-1}(y)}. \quad (62)$$

The exact transition is

$$K_t^*(x_{t-1} | x_t) = K_t(x_{t-1} | x_t) \frac{h_{t-1}(x_{t-1})}{h_t(x_t)}. \quad (63)$$

Proposition 3 (One-step transition error). *Assume for a fixed parent x_t that*

$$e^{-\epsilon} \leq e_{t-1}(y) \leq e^\epsilon \quad \text{for all children } y \text{ reachable from } x_t. \quad (64)$$

Then

$$\text{TV}(\hat{K}_t(\cdot | x_t), K_t^*(\cdot | x_t)) \leq \frac{e^{2\epsilon} - 1}{2}. \quad (65)$$

For small ϵ , this is $O(\epsilon)$.

Proof. For a child y ,

$$\frac{\hat{K}_t(y | x_t)}{K_t^*(y | x_t)} = \frac{\hat{h}_{t-1}(y)}{h_{t-1}(y)} \cdot \frac{h_t(x_t)}{\sum_z K_t(z | x_t) \hat{h}_{t-1}(z)} \quad (66)$$

$$= e_{t-1}(y) \cdot \frac{\sum_z K_t(z | x_t) h_{t-1}(z)}{\sum_z K_t(z | x_t) h_{t-1}(z) e_{t-1}(z)}. \quad (67)$$

The first factor lies in $[e^{-\epsilon}, e^\epsilon]$ and the reciprocal-normalization factor also lies in $[e^{-\epsilon}, e^\epsilon]$. Thus the density ratio lies in $[e^{-2\epsilon}, e^{2\epsilon}]$, which yields the stated total-variation bound. $\square$

12.3 Why Bellman calibration matters

A clean-prediction verifier score can be locally useful but temporally inconsistent. Suppose $\hat{h}^{(0)}$ is biased. Bellman backup reduces the component of error that violates the transition equation. In operator notation, let

$$(\mathcal{K}_t f)(x_t) = \sum_y K_t(y | x_t) f(y). \quad (68)$$

The true function satisfies

$$h_t = \mathcal{K}_t h_{t-1}. \quad (69)$$

A calibrated estimator should make the residual

$$r_t = \log \hat{h}_t - \log \mathcal{K}_t \hat{h}_{t-1} \quad (70)$$

small. This residual is directly measurable by Monte Carlo and can be used both as a diagnostic and as an adaptive compute trigger.

12.4 Variance intuition

Under an exact Doob transform, the terminal reward potential is absorbed into the transition kernel. This removes the need for large terminal importance weights. In fact, under the exact transformed process,

$$\prod_{t=T}^1 \frac{h_{t-1}(X_{t-1})}{h_t(X_t)} = \frac{G(X_0)}{h_T(X_T)}. \quad (71)$$

The telescoping structure is the source of variance reduction: instead of discovering high reward only at the end, the process continuously steers toward states with high expected terminal reward.

13 Practical estimator design

13.1 Verifier choices

V can be:

- a process reward model for math reasoning;
- a lightweight reference-free verifier as in search-based DLM scaling;
- a symbolic checker for code, arithmetic, or constraints;
- a self-consistency score over multiple clean predictions;
- a hybrid score combining answer reachability, formatting validity, and local consistency.

The method does not require training a new model. It only requires a terminal potential usable at inference time.

13.2 Top- K approximation over tokens

The exact position score (36) sums over the full vocabulary. In practice, use the top- K token set $\mathcal{T}_i$ under the base denoiser:

$$\hat{H}_i(x_S) = \sum_{v \in \mathcal{T}_i} p_\theta(v \mid x_S, i, c) \hat{h}(x_S^{i=v}). \quad (72)$$

A simple bias control is to keep the remaining mass as a baseline:

$$\hat{H}_i^{\text{tail}}(x_S) = \sum_{v \in \mathcal{T}_i} p(v) \hat{h}(x_S^{i=v}) + \left(1 - \sum_{v \in \mathcal{T}_i} p(v)\right) \hat{h}_{\text{base}}(x_S). \quad (73)$$

13.3 Progressive depth schedule

Early in denoising, states are highly masked and h_t is hard to estimate. Later, states are closer to final outputs and clean prediction is more reliable. A natural depth schedule is

$$D_t = D_{\min} + \left\lceil D_{\max} \left(1 - \frac{|S_t|}{L}\right)^\alpha \right\rceil, \quad (74)$$

or, more adaptively,

$$D_t(x) = \min \left\{ D_{\max}, D_{\min} + \left\lceil \gamma \hat{\mathcal{E}}_t(x) \right\rceil \right\}. \quad (75)$$

13.4 Exploration vs exploitation

Use a temperature τ in (53). Low τ follows the strongest harmonic edge; higher τ maintains diversity. Another robust choice is ϵ -mixture sampling:

$$\tilde{K}_t = (1 - \epsilon) \hat{K}_t^{\text{Doob}} + \epsilon K_t. \quad (76)$$

This prevents collapse if $\hat{h}$ is noisy.

14 Implementation plan

14.1 Minimal implementation

The first implementation should avoid unnecessary complexity:

1. Use one particle or a small population.
2. At each step, select a candidate pool of positions: top confidence, top entropy, and random positions.
3. For each candidate position, evaluate top- K child tokens.
4. Estimate $\hat{h}$ by clean prediction plus one Bellman backup.
5. Choose the position/token by Doob score.

This minimal version tests whether harmonic ordering beats confidence ordering at matched network evaluations.

14.2 Full implementation

The full version uses:

- population of particles;
- dynamic backup depth by residual;
- optional SMC resampling when ESS is low;
- block proposals for speed;
- cached clean predictions and verifier scores.

14.3 Compute accounting

Let:

- T be denoising steps;
- B candidate children per state;
- M rollout children per backup;
- D backup depth;
- N particles.

A rough NFE cost is

$$\text{NFE} \approx NT(1 + B + BMD), \quad (77)$$

where caching and shared batched evaluation can reduce constants. The key comparison should always be at matched NFE against S^3 , PG-DLM, SMC, and confidence-based denoising.

15 Experiments

15.1 Main benchmarks

Use benchmarks where reward-guided DLM test-time scaling is meaningful:

- **GSM8K**: arithmetic and chain-of-thought reasoning.
- **MATH-500**: harder math reasoning.
- **Countdown**: search-like arithmetic composition.
- **ARC-Challenge**: multi-choice reasoning.
- **TruthfulQA**: verifier-guided factuality/truthfulness.
- **HumanEval** / **MBPP**: code generation with executable reward.

15.2 Baselines

Compare against:

1. Base DLM decoding.
2. Confidence / margin-based denoising order.
3. Best-of- K .
4. SMC / Feynman–Kac steering.
5. PG-DLM.
6. S^3 .
7. Ablated HDO-DLM with clean prediction only, no Bellman backup.
8. Full HDO-DLM with Bellman calibration and adaptive residual compute.

15.3 Primary metrics

- Accuracy or pass@1 vs NFE.
- Verifier reward vs NFE.
- Perplexity or base-model likelihood preservation.
- Bellman residual vs accuracy.
- ESS if using population resampling.
- Denoising-order statistics: which positions are selected earlier than confidence would choose?
- Premature commitment rate: how often answer tokens are fixed before reasoning-critical tokens become stable.

15.4 Most important experiment

The decisive experiment is:

Replace S^3 's raw clean-prediction frontier score with a Bellman-calibrated h estimate and use the resulting Doob edge score for denoising order. At matched NFE, measure whether performance improves and whether the improvement correlates with reduced martingale residual.

This directly tests the thesis that the missing ingredient is not simply more search but a better approximation to the backward information function.

16 Ablations

1. **No Bellman backup:** $\hat{h} = \exp\{\beta V(\hat{x}_0)\}$ only.
2. **No Doob ordering:** use $\hat{h}$ only for final resampling, not transition selection.
3. **Confidence-only:** choose highest confidence token.
4. **Reward-only greedy:** choose child with highest immediate verifier clean prediction, without K_t prior or harmonic calibration.
5. **Residual-adaptive compute vs fixed compute:** test whether residual-triggered backups allocate compute better.
6. **Top- K token size:** $K = 1, 2, 4, 8$.
7. **Backup depth:** $D = 0, 1, 2, 4$.
8. **Exploration temperature:** greedy vs sampled Doob edges.

17 Expected findings and failure modes

17.1 Expected findings

We expect HDO-DLM to help most when:

- final correctness depends on a few low-confidence but high-impact reasoning tokens;
- confidence-based schedules commit easy filler tokens too early;
- verifier lookahead is useful but noisy and benefits from temporal calibration;
- search-based methods waste compute on frontier states whose apparent score is not Bellman-consistent.

17.2 Failure modes

- **Verifier noise:** if V is badly miscalibrated, $\hat{h}$ may guide the sampler toward verifier hacks.
- **Expensive backups:** deeper harmonic estimation may cost more than it helps.
- **Top- K bias:** important low-probability tokens may be absent from candidate sets.
- **Over-sharp β :** very large reward temperature can produce brittle transitions and mode collapse.
- **Long-horizon error accumulation:** small h errors can compound over many denoising steps.

18 Strongest reviewer criticism

Criticism 1: “This is just S^3 with more rollouts.” No. S^3 uses verifier-guided lookahead as a frontier score for resampling. HDO-DLM treats verifier lookahead as an approximation to a specific mathematical object, h_t , and enforces the Bellman equation that this object must satisfy. The transition rule is not arbitrary resampling; it is the approximate Doob kernel

$$\hat{K}_t(y | x) \propto K_t(y | x) \hat{h}_{t-1}(y). \quad (78)$$

The ablation “clean prediction only vs Bellman-calibrated h ” is designed to isolate this difference.

Criticism 2: “Estimating h_t is intractable.” Exact h_t is intractable; that is precisely why the method is a test-time scaling algorithm. The proposal is not to compute h_t globally, but to estimate it locally on candidate edges that the DLM already proposes. This is analogous to using local value estimates in search rather than solving a full dynamic program.

Criticism 3: “Doob transform is known; where is the novelty?” The novelty is not the Doob transform itself. The novelty is applying it to the DLM mask lattice to derive a denoising-order rule. In autoregressive LMs, the next textual position is fixed. In DLMs, the next edge on the mask lattice is a decision. HDO-DLM shows that the reward-conditioned optimal decision is a harmonic-flow decision.

Criticism 4: “Why not train a twist function?” Training a twist function is a valid future extension. But the proposal targets training-free test-time scaling, matching the regime of PG-DLM and S^3 . Bellman-calibrated verifier rollouts give a way to approximate the twist without additional training.

19 Why this could be a strong research direction

The broader claim is:

$$\boxed{\text{DLM inference-time scaling is harmonic inference on a mask lattice.}} \quad (79)$$

This view unifies several phenomena:

- Reward guidance becomes terminal-potential conditioning.
- The ideal sampler is a Doob-transformed denoising chain.

- Denoising order is induced by harmonic future-reward flow.
- Test-time compute improves inference by reducing error in h_t .
- Adaptive compute is triggered by Bellman/martingale residual.

This gives a principled story that is more than an engineering combination. PG-DLM contributes reference-preserving trajectory refinement. S^3 contributes diversity-preserving verifier-guided frontier search. HDO-DLM contributes Bellman-calibrated Doob ordering: a mathematically exact answer to what the reward-conditioned DLM transition should be.

20 Possible paper outline

1. **Introduction:** DLMs expose denoising order; confidence/order heuristics are not reward-optimal.
2. **Background:** DLM path measures, reward-tilted inference, SMC/PG-DLM/ S^3 .
3. **Theory:** backward information function, harmonic equation, Doob transform.
4. **Method:** HDO-DLM candidate expansion, Bellman-calibrated h estimation, Doob edge selection.
5. **Analysis:** exactness with exact h , approximation error, martingale residual.
6. **Experiments:** reasoning, code, truthfulness; matched-NFE comparisons and ablations.
7. **Discussion:** limitations, verifier noise, future training of twist functions.

21 Summary

The central insight is simple:

The correct reward-conditioned DLM sampler is already known: it is the Doob h -transform.

(80)

For masked DLMs, this transform has an immediate interpretation as a denoising-order rule. The next token or block should be selected not by local confidence, but by the expected terminal reward potential of the child state. Therefore, the right test-time scaling axis is the accuracy of the backward information estimate h_t .

References

- [1] Meihua Dang, Jiaqi Han, Minkai Xu, Kai Xu, Akash Srivastava, and Stefano Ermon. Inference-Time Scaling of Diffusion Language Models via Trajectory Refinement. *arXiv:2507.08390*, 2025.
- [2] Ahsan Bilal, Muhammad Ahmed Mohsin, Muhammad Umer, Asad Aali, Muhammad Usman Khanzada, Muhammad Usman Rafique, Zihao He, Emily Fox, and Dean F. Hougen. S^3 : Stratified Scaling Search for Test-Time in Diffusion Language Models. *arXiv:2604.06260*, 2026.
- [3] Muhammad Hasan et al. Discrete Feynman–Kac Correctors. *OpenReview / arXiv:2601.10403*, 2026.
- [4] Stephen Zhao, Rob Brekelmans, Alireza Makhzani, and Roger Baker Grosse. Probabilistic Inference in Language Models via Twisted Sequential Monte Carlo. *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [5] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution. *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [6] Jeremy Heng, Adrian N. Bishop, George Deligiannidis, and Arnaud Doucet. Controlled Sequential Monte Carlo. *The Annals of Statistics*, 48(5), 2020.
- [7] Nick Whiteley and Anthony Lee. Twisted Particle Filters. *The Annals of Statistics*, 42(1), 2014.
- [8] Jiaqi Chang et al. Training-Free Adaptation of Diffusion Models via Doob’s h -Transform. *arXiv:2602.16198*, 2026.
- [9] Christophe Andrieu, Arnaud Doucet, and Roman Holenstein. Particle Markov Chain Monte Carlo Methods. *Journal of the Royal Statistical Society: Series B*, 2010.